Process-Level Feedback and Retrieval Quality in Agentic Information Search

Sergey Sedov and Adhiraj Singh and Vedant Jagtap
New York University
ss19021@nyu.edu, as19687@nyu.edu, vsj7589@nyu.edu

Abstract

We explore how verifier feedback and retrieval design choices affect the quality of Agentic Information Retrieval on multi-hop question answering. Current benchmarks often prioritize end-to-end task completion, treating the agent's intermediate reasoning as a black box, yet process-level decisions strongly shape the final answer. We experiment on the synthetic multi-hop Q&A dataset and index composed of YouTube video transcripts, Lex Fridman's podcasts specifically. We compare performance between retrieval tools (grep), varying number of retrieval steps and evaluating with or without process feedback that is provided in-context from LLM-as-a-Judge. We find that simple grep (Exact-Match) retrieval is competitive with BM25 and embedding search, benefits strongly from added retrieval steps, and shows gains that align with lower hallucination rates. Process feedback further reduces grep hallucination, while the strongest overall configuration is embedding retrieval with process feedback at maximum depth. Addressing low-quality chunk retrieval with simple methods, we propose to use query-independent chunk quality scores. However our experiments do not show consistent improvements with them. Finally, we study the effect of pre-knowledge-cutoff contamination. Surprisingly, we find that model's performance on older examples is significantly worse than on the newer portion of them, which suggests the strong effect of time-dependent distribution drift.

1 Introduction

Retrieval-augmented generation (RAG) was introduced to reduce factual errors by grounding generation in retrieved evidence (Lewis et al., 2020; Guu et al., 2020). In practice, many current systems no longer run as one-shot pipelines. They run as *agents*: they make multiple retrieval calls by querying retrieval tools, revise search queries, and decide when to stop and answer. This shift from static RAG to agentic RAG is now widely discussed in recent surveys and benchmark work (Schick et al., 2023; Yao et al., 2023; Asai et al., 2024; Lin et al., 2025). However, most reports still emphasize endpoint metrics and under-document the retrieval process that produced those endpoints.

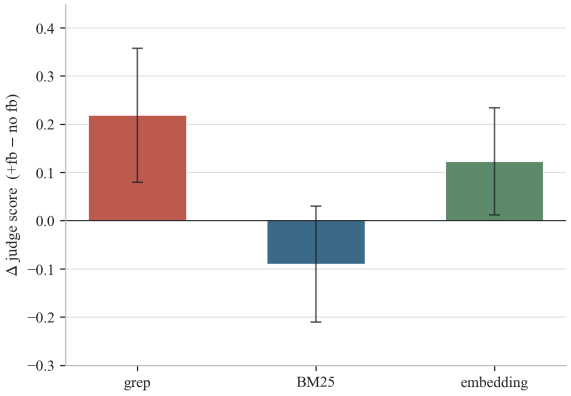

Figure 1: The effect of applying process-level LLM-as-a-Judge feedback to Agentic retrieval. We observe significant difference in performance gains across various retrieval modules. Simple grep retrieval benefits from the provided feedback the most, BM25 does not show any improvement at all, while embedding-model based retrieval shows intermediate gains.

That gap matters for real deployment. A system can score well on final correctness while still showing poor retrieval behavior, high hallucination risk, or excessive latency/cost. Conversely, a system with slightly lower aggregated score can be much more stable and interpretable under process-level diagnostics.

We start by studying the effect of various modifications to the baseline agentic RAG pipeline, since these effects remain poorly understood. We compare how agentic RAG performs on question-answering dataset when different retrieval methods are used with various budget of retrieval steps. Further on, we evaluate the effect of adding process feedback that is provided in context from an-

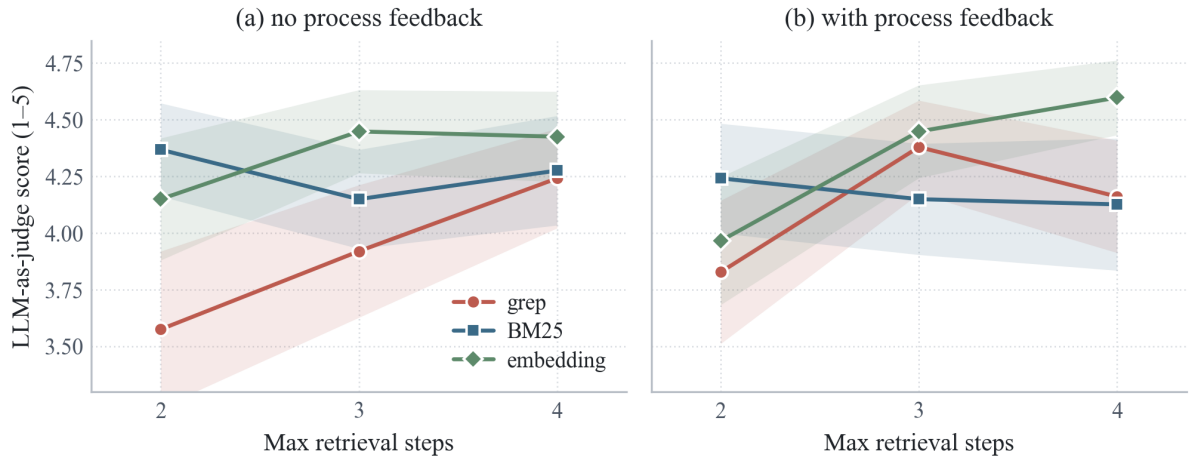

Figure 2: LLM-as-judge score as a function of max retrieval steps. Left panel: no process feedback. Right panel: with process feedback. Embedding remains the strongest curve in most settings. On multi-step retrieval or with process feedback grep shows results comparable to other methods. BM25 slightly degrades either from multiple steps or from process feedback. Embedding-model based retrieval performs the best. Overall, number of iterations affects the final performance more than process feedback.

other language model that is prompted to point out retrieval inconsistencies. Our hypothesis is that generator-verifier gap exists: the same model family that generates final answers may still benefit from intermediate verifier-style feedback during the search loop.

We also evaluate a second hypothesis: query-independent chunk quality scores, motivated by improving robustness against noisy chunks retrieval when simple retrieval tools are used. This motivation comes directly from a practical mismatch between web search and local RAG. Web search benefits from mature quality signals such as PageRank and long-standing authority heuristics. In contrast, local transcript indices are usually ranked only by lexical or embedding similarity. That setup can retrieve semantically similar but low-value chunks (for example, sponsor segments, repeated intros, or broad topical fragments). In our proposal framing, this is exactly where verifier feedback and offline chunk quality signals could matter most: not in replacing retrieval, but in correcting retrieval noise before it contaminates generation.

To keep the report self-contained, we summarize our final empirical setup directly here. We run a full 36-condition grid over 87 synthetic Q&A items (3132 evaluations total), varying retriever (`grep`, `bm25`, `embedding`), max retrieval depth (2/3/4), process feedback (off/on), and chunk quality scores (off/on).

Research questions. We organize the final analysis around four concrete questions:

- **RQ1:** Does process feedback improve quality and reduce hallucination consistently?
- **RQ2:** How much can lexical retrieval (`grep`) close the gap to stronger retrievers when step budget is increased and process feedback is applied?
- **RQ3:** Do query-independent chunk quality scores provide robust gains?
- **RQ4:** How does agentic RAG performance vary between pre- and post-knowledge-cutoff examples, and does the old-era portion show any sign of parametric-knowledge leakage?

Main findings. First, embedding retrieval is strongest overall on quality indicators and appears most robust across condition families. Second, grep improves substantially with depth, and its hallucination rate drops sharply from low- to high-step settings. Third, process feedback is useful in many cells but not a universal win. Fourth, quality scores yields mixed effects and require careful design in the future. Lastly, post-cutoff items are easier, not harder, in this benchmark.

Contributions. This paper contributes: (1) a complete 36-condition evaluation of process-level retrieval decisions on a nontrivial Q&A benchmark; (2) a controlled comparison of retriever, depth, feedback, and quality-reweight interactions; (3) a

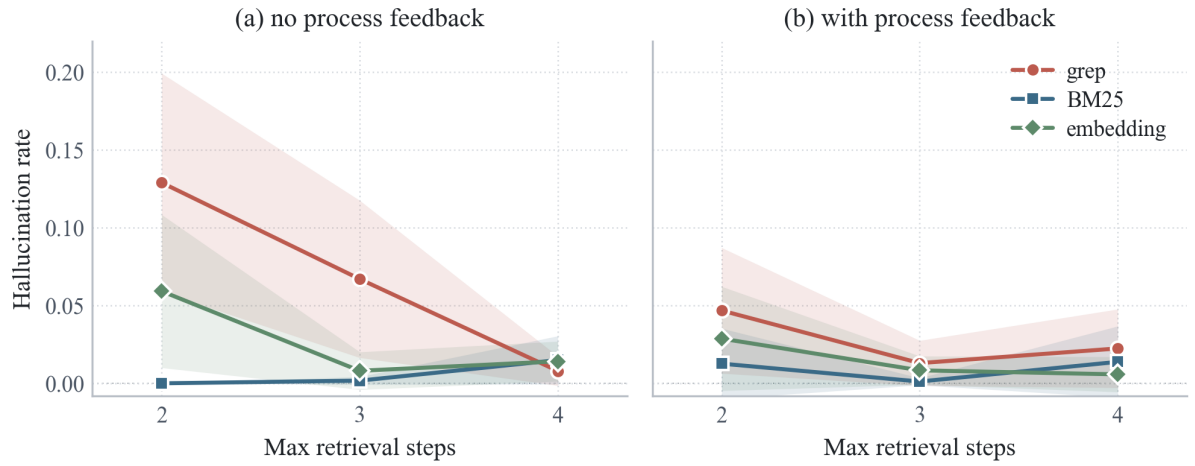

Figure 3: Hallucination rate vs max retrieval steps, split by no feedback (left) and process feedback (right). Grep starts with the highest hallucination at 2 steps and drops sharply with additional steps. Process feedback significantly improves hallucination rate, which is especially important for the performance of grep retrieval. We attribute hallucination rate drop with the performance improvements observed when process-feedback is used together with grep retrieval.

careful treatment of mixed and negative results, including where the original hypotheses do *not* hold cleanly. This directly preserves the proposal's three intended novelty axes: process-vs-outcome evaluation, retrieval-side quality signaling for raw transcript corpora, and contamination-aware analysis using era-aware slicing.

2 Related Work

2.1 From static RAG to agentic retrieval loops

Classical RAG pipelines established the importance of retrieval-grounded generation for factual tasks (Lewis et al., 2020; Guu et al., 2020). More recent work moved from one-shot retrieval toward iterative or tool-augmented behavior, where the model can decide what to retrieve and when (Schick et al., 2023; Yao et al., 2023). This line is closely related to modern agentic RAG formulations and surveys (Asai et al., 2024; Lin et al., 2025; **?**). The practical implication is that evaluation should not treat retrieval as a fixed pre-step; it is now part of the reasoning trajectory.

This shift is central to our proposal lineage: in one-shot evaluation, the retrieval stage is often treated as a frozen preprocessing step. In agentic settings, retrieval becomes a repeated decision process. Each iteration changes the context seen by the model and therefore changes subsequent retrieval choices and final synthesis behavior. That is why a process-level study is necessary even when endpoint metrics are already available.

2.2 Multi-hop Q&A benchmarks

Multi-hop question answering has been studied extensively on benchmarks such as HotpotQA (Yang et al., 2018), 2WikiMultiHopQA (Ho et al., 2020), and MuSiQue (Trivedi et al., 2022), which construct questions requiring composition across multiple passages. Our synthetic dataset applies the same multi-hop principle to a single-creator transcript corpus, where 'hops' often correspond to cross-episode reasoning rather than cross-document entity linking.

2.3 Process supervision and intermediate verification

Process-level supervision has shown gains over outcome-only supervision in complex reasoning settings (Lightman et al., 2023). While this evidence primarily comes from mathematical reasoning, the underlying principle is relevant to retrieval loops: if intermediate steps can be critiqued, downstream generation may become more reliable. Our project operationalizes this idea in an information-search context by injecting process feedback between retrieval steps rather than only scoring final outputs.

2.4 LLM-as-a-judge: usefulness and limits

LLM-as-a-judge is now common for open-ended evaluation because it scales better than exhaustive human annotation (Zheng et al., 2023). At the same time, recent work highlights failure modes

such as position bias, verbosity bias, and reference sensitivity (Krumdick et al., 2026). We account for this literature in two ways: we report multiple metrics (not judge score alone), and we explicitly treat judge-based conclusions as probabilistic evidence rather than absolute ground truth.

2.5 Faithfulness and hallucination in RAG

A growing body of work evaluates faithfulness directly in RAG settings, including benchmark and leaderboard frameworks for hallucination-sensitive evaluation (Tamber et al., 2025). A recurring finding is that retrieval quality and final answer faithfulness are related but not identical; models can retrieve relevant text and still synthesize unsupported claims. Our design follows this insight by tracking retrieval precision, reference recall, and hallucination alongside endpoint success.

2.6 Positioning of this work

Unlike prior work that reports endpoint metrics on agentic RAG, we run a controlled grid of experiments that isolate the interaction between retriever, depth, and process feedback.

3 Methodology

3.1 Agentic Retrieval

The agent starts with a system prompt, tool description, and question. At each step it produces a short reasoning trace and a retrieval tool call; the backend returns top-k chunks. When feedback is enabled, a judge LLM then critiques the chunks. In later steps the agent sees only prior chunks and judge feedback — its own past reasoning is dropped. The agent is told the step budget upfront and must emit a final answer once it is exhausted.

3.2 Dataset and Indexing

We construct a local search index consisting of approximately 450 Lex Fridman podcast transcripts. This includes ∼350 auto-generated transcripts that are already published as datasets and 100+ high-fidelity handwritten transcripts from the official website. Additionally, we separate dataset into three parts, by model knowledge cut-off date: old, mixed and new episodes. This allows us to analyze the effect of contamination in case of YouTube transcripts data.

3.2.1 Synthetic Q&A Generation

To evaluate the system, we generate a set of 87 Question-Answer pairs, balanced by the time-era that underlying episodes come from. Our approach is to manually select topics, utilizing LLMs on summaries of episodes with pre-defined retrieval guidelines. Thus, we assume that this methodology would provide challenging tasks for RAG evaluation on index created from raw chunked data. We use Gemini 2.5 Flash to analyze the full text of episodes and their summaries and generate multi-hop questions that require information found in specific chunks. We focus on "cross-episode" queries, such as: "How does the guest in episode A's view on artificial consciousness contrast with the guest in episode B?" or "What recurring argument about the Fermi Paradox does Lex bring up across his interviews with physicists?". The following section describes dataset generation in detail.

3.3 Agentic RAG Dataset Construction Algorithms

Let $\mathcal{C}$ be the corpus of full transcripts, and $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$ be the set of high-level summaries for each episode. We specifically extract metadata regarding Guests $\mathcal{G} = \{G_1, G_2, \ldots, G_n\}$, entities E and details D. Entities are unique identifiers (Person, Project, or Concept) that exists in $\mathcal{S}_i$ and serves as a search key to retrieve $\mathcal{C}_j$. Details are specific, high-granularity facts or perspectives within $\mathcal{S}$. We utilize high-reasoning LLM used for automated topic selection and question generation, based on manually selected inputs to algorithms. For each question type, we define the Ground Truth (GT) as the tuple (Q, A, Ref), where A is the final answer and Ref is the set of required transcript segments.

3.3.1 Type 1: Multi-Hop Bridge Questions

Objective: Create a dependency where Episode j can only be identified by a specific entity E found in Episode i. These questions require the agent to find one entity to unlock the next. The agent cannot find the final answer without first retrieving and processing an intermediate piece of information.
Example:

- **Question:** "What does the creator of the 'Atlas' humanoid robot think about the 'uncanny valley' effect?"
- **Step 1 (Iterative):** Search 'Atlas robot' → Identify Marc Raibert (Episode 42).
- **Step 2 (Iterative):** Search 'Marc Raibert uncanny valley' → Retrieve the answer.
- **Answer:** He believes it is a distraction and focuses on functional movement over human like-

ness.

Algorithm:

1: **Input:** Episode Summaries $\mathcal{S}$, Guest G.
2: **Select:** $s_i, s_j \in \mathcal{S}$ where s_i contains entity E_i associated with the guest G, and s_j is one of the interviews of the same guest where they mentioned detail D.
3: **Prompt:** "Formulate Q asking for a detail $D \in s_j$ by referencing $E_i \in s_i$ without naming the guest."

3.3.2 Type 2: Comparative Viewpoint

The objective is to find a shared topic θ discussed by Guest G_1 and Guest G_2.

Example:

- **Question:** "Contrast Yann LeCun's and Andrej Karpathy's views on the necessity of 'world models' for AGI."
- **Step 1:** Retrieve LeCun on LLM limitations.
- **Step 2:** Retrieve Karpathy on Software 2.0/LLM scaling.
- **Answer:** LeCun argues LLMs lack world models; Karpathy views them as a foundational OS that can simulate reasoning via scale.

Algorithm:

1: **Input:** Episode Summaries $\mathcal{S}$, guests G_1, G_2, shared topic θ.
2: **Select:** $\{s_i, s_j\}$ where both guests discuss topic θ but reach different conclusions.
3: **Prompt:** "Identify the differences in opinions between Guest A and Guest B regarding θ."

3.3.3 Type 3: Temporal Evolution

The objective is to track a change in a guest's position over time t. Agent needs to understand total amount of episodes and handle metadata in retrieval queries.

Example:

- **Question:** "How did Elon Musk's timeline for a crewed Mars mission change between his 2021 and 2024 interviews?"
- **Step 1:** Search 'Elon Musk Mars 2021'
- **Step 2:** Search 'Elon Musk Mars 2024'
- **Answer:** The timeline became more aggressive, shifting from a 5–10 year window to 3–5 years.

Algorithm:

1: **Input:** Summaries $\mathcal{S}$ for Guest G_k at $t_1, t_2, \ldots, t_n$.
2: **Select:** A claim C that shifted between t_{early} and t_{late}.
3: **Prompt:** "How did Guest G_k's prediction for [Event] change between their first and most recent appearance?"

3.3.4 Type 4: Quantitative Aggregation

Objective: Force exhaustive retrieval across the entire corpus to find an extremum.

Example:

- **Question:** "Among all AI safety experts interviewed, who gave the most pessimistic (lowest) probability for human survival post-AGI?"
- **Step 1:** Search 'Probability human survival AGI'.
- **Step 2:** Aggregate values (e.g., Eliezer Yudkowsky: 1%, Max Tegmark: 20%).
- **Answer:** Eliezer Yudkowsky, with a near-zero probability.

Algorithm:

1: **Input:** All Summaries $\mathcal{S}$, Topic θ.
2: **Select:** Looking at summaries, identify all videos where topic θ was mentioned. Define a common metric M associated with topic θ (e.g., probability, years, percentage). Lower down the scope of considered videos.
3: **Prompt:** "Which guest assigned the lowest probability to the 'alignment problem' being solved by 2050?"

3.4 Retrieval Modules

We implement and compare three distinct retrieval mechanisms:

1. **Grep (Exact-Match):** A baseline utilizing basic sub-string matching. Agent needs to reformulate the question to retrieve all relevant chunks, prompting the retrieval tool with small strings to find matching chunks.

2. **BM25:** A probabilistic retrieval model that ranks documents based on term frequency and inverse document frequency. The most common approach used in RAG pipelines.

3. **Embedding-Based Retrieval:** A neural approach using vector embeddings produced by a separate embedding model, calculating cosine similarity between the query and the text embeddings to assign similarity scores. We use default Google text embedding model.

3.5 Agent and Verifier Configurations

We study the effect of in-context process feedback provided from LLM-as-a-Judge in agentic RAG.

Judge is the same LLM prompted separately to verify mistakes or inconsistencies that retrieved chunks have with respect to the question.

1. **No Judge:** Agent performs an iterative search (using tools to call the retrieval backends) and generates an answer once it deems it has enough information.

2. **Judge-in-Process:** After each retrieval call, the verifier reviews the retrieved chunks. The verifier provides natural language feedback to the agent (e.g., "Chunk 2 is an advertisement; ignore it," or "The retrieved info is too broad and does not answer the whole question; try searching for specifically X").

3.6 Chunk Quality Metrics

We consider using LLM verification during the index processing stage. We pre-process chunk quality scores for index, without performing these computations on inference time, implementing a simple version of LLM-based chunk quality scores. LLM is prompted to score individual chunks on a scale for "Integrity" (clarity and lack of transcription errors), "Information Density" (the ratio of meaningful content to filler words or ads), and, optionally, "Dataset Relevance" (whether this chunk is likely to appear relevant to Q&A dataset). The last score extends quality scores to the specifics of each evaluation dataset, as LLM is provided with the general description of Q&A dataset.

4 Experimental Setup

4.1 Condition grid

We evaluate:

- retriever: `grep`, `bm25`, `embedding`,
- max retrieval steps: 2, 3, 4,
- process feedback: off/on,
- quality scores: off/on.

This yields $3 \times 3 \times 2 \times 2 = 36$ conditions. Each condition is evaluated on all 87 Q&A items.

4.2 Models and implementation details

We use the same Gemini models for agentic retrieval, question-answering and judging, and Gemini embeddings for vector retrieval. Core settings follow the progress pipeline and final run scripts:

- generative model: Gemini 2.5 Flash,
- embedding model: Gemini embedding 001,

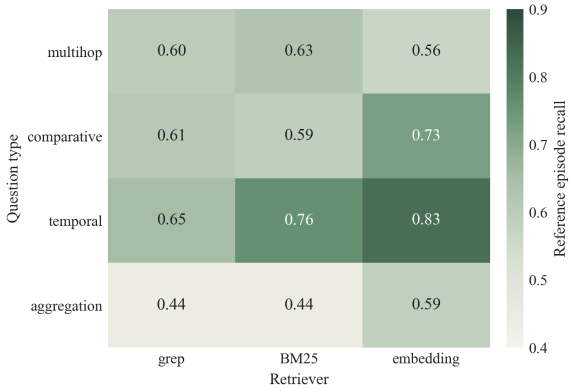

Figure 4: Reference-episode recall by Q&A type and retriever. Embedding is strongest on comparative and temporal questions; multihop is more balanced. Embedding-model based retrieval deals better with aggregation tasks, while agents using grep and BM25 struggle with it.

- retrieval per step: top-$k = 5$ chunks,
- chunking: 1000 characters with 150-character overlap.

4.3 Evaluation metrics

Following the original proposal, we use a multi-metric evaluation design so that no single score is treated as sufficient evidence of quality. For each condition/example pair we record:

- LLM-as-judge score (1–5): integer from a Gemini 2.5 Flash call over question, answers, and retrieved chunks,
- retrieval precision: judge-emitted fraction of retrieved chunks marked relevant,
- hallucination rate: judge-emitted fraction of the answer unsupported by retrieved chunks,
- reference-episode recall: $|R \cap F|/|R|$ over reference episodes R and episodes F seen in the trajectory,
- lexical F1 and exact match,
- success (binary): $F1 \geq 0.55$ or classified as correct by Judge,
- cost and latency

4.4 Reference-episode recall by Q&A type and retriever

5 Results

5.1 Global outcomes

Figure 1 previews the central pattern: process-level feedback does not uniformly help every retriever.

Grep benefits the most, embedding shows intermediate gains, and BM25 is worse / unchanged given the confidence interval. This retriever-dependent behavior motivates framing the study around interaction effects rather than a single best configuration.

5.2 Judge-score trends by retriever, depth, and feedback

Figure 2 shows two practical effects. First, embedding dominates most cells on judge score, especially in feedback-enabled settings at 4 steps. Second, grep benefits strongly from added depth, reducing the gap versus BM25 and embedding in middle and high-depth settings. This supports a nuanced claim: grep is not globally best, but it can become competitive when depth and feedback are configured carefully.

5.3 Hallucination trends

Figure 3 presents evidence for a key project takeaway: for grep, both deeper retrieval and process feedback substantially lower hallucination. Hallucination drops from 13% (2 steps) to 6.5% (3 steps) to 1% (4 steps) with no judge, while adding a judge on the same 2-step retrieval improves hallucination rate from 13% to 5%. Figure 4 helps explain why embedding leads in many quality metrics: it retrieves gold-reference episodes more consistently for complex Q&A types. Aggregation remains the hardest family in general, with neural embedding retrieval significantly outperforming both grep and BM25.

5.4 Quality chunk scores

We use LLM-scored chunks in [0,1] aggregated as $0.2 \cdot \text{Integrity} + 0.3 \cdot \text{Density} + 0.5 \cdot \text{Relevance}$, with ads hard-filtered. Retrieval ranks by $0.7 \cdot \text{Similarity} + 0.3 \cdot \text{Quality}$. Figure 5 shows no consistent improvement. We attribute this to the low noise level of Lex Fridman transcripts, where ads are already removed and most chunks carry useful content. Quality reweighting likely matters more on noisier corpora.

5.5 Knowledge-cutoff effect

We split the 87 items by reference-episode era relative to the January 2025 cutoff. Results are presented in Figure 6. Contrary to the contamination hypothesis, old-era items score lower (success 0.72 vs. 0.80; recall 0.54 vs. 0.68, $p = 0.019$). We attribute this effect to topic distribution drift in newer

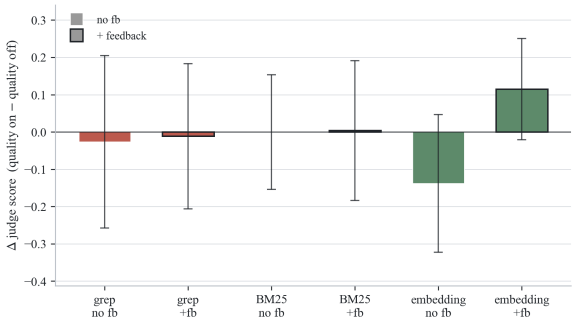

Figure 5: Difference between grouped runs with and without chunk quality scores. We observe no significant difference in all considered experiments.

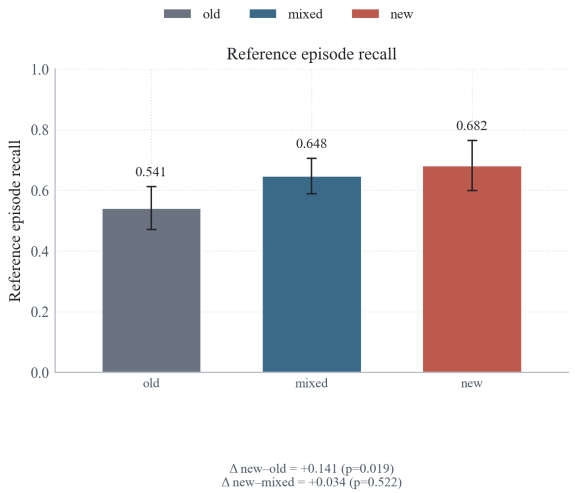

Figure 6: Reference-episode recall by sample era: Q&A samples created from old, mixed and new episodes by January 2025 knowledge cut-off date. Unlike our initial hypothesis that model might cheat by using pre-trained knowledge on old samples, we observe an inverse effect - it performs significantly better on the newer samples.

Lex Fridman podcasts (potentially more narrow Q&A samples).

6 Discussion

6.1 Process feedback hypothesis

The proposal hypothesis was that process-level verification could improve agent behavior. Our final answer is: *yes, but conditionally*. Feedback improves outcomes in many settings, especially where retrieval noise is high, but it is not a guaranteed gain in every retriever-depth combination. This is a key practical result: process feedback should be treated as a tunable policy, not a universal default. Empirical evidence shows that increasing the number of retrieval steps has stronger effect on the final performance, nevertheless process feedback contribution is statistically significant.

6.2 How the findings connect to prior work

Prior process-supervision work suggests that intermediate critique can improve reasoning and help web search. Our results align with that direction but add a retrieval-specific nuance: improvements depend on retrieval backend and depth. The findings also support the broader RAG observation that retrieval relevance and final answer quality are related but not identical.

6.3 Negative results and what they contribute

The quality chunk scores arm does not deliver a consistent improvement across the full grid. Simple query-independent chunk scoring is not enough by itself in this setup. We attribute this largely to the low level of noise in the Lex Fridman corpus; quality reweighting has little bad content to reject. The result narrows the design space for future work and motivates better calibration or learned weighting strategies.

7 Limitations and Future Work

Several limitations bound the scope of our conclusions. First, we evaluate a single corpus and index (Lex Fridman transcripts); robustness to different data scales, noise levels, and Q&A styles remains to be shown. Second, our evaluation set of 87 synthetic Q&A items is modest, and subgroup analyses (era, Q&A type) are further constrained by cohort imbalance. Third, we report a single run per condition — variance intervals come from bootstrapping over items, not from seeded reruns of the agent itself. Fourth, our chunk-quality experiments explore only one design (a fixed linear combination of three LLM-scored axes) under tight compute limits; richer formulations (learned weighting, reranker-style reweighting, or TF-IDF-style relative scoring within documents) are left for future work. Finally, the judge and the generator come from the same model family (Gemini 2.5 Flash), which raises the risk of evaluator bias.

A further concern is that Gemini 2.5 Flash is used for Q&A generation, answer generation, and judging. We partially mitigate this at dataset construction through a structured pipeline: ground-truths are produced stepwise from curated summaries, guest metadata, and entity lists rather than free-form corpus access, with each stage constrained to a specific role. Fully disentangling generator bias would still require cross-family generation and adjudication.

8 Conclusion

In this work we presented empirical evidence on the importance of several hyperparameters in iterative agentic information retrieval setting. To evaluate it properly, we presented a new Q&A dataset, questions of which require multiple retrieval tries to combine information from different sources. Most importantly, we showed that iterative retrieval and process judge feedback improve basic grep retrieval quality to the levels comparable with BM25 and embedding-model retrieval modules. This is an important finding as in many applied scenarios index pre-processing is unfavorable and raises a practical question about retrieval quality trade-off between search tools. For instance, when underlying index changes significantly between agent calls, its pre-processing can become a hurdle that grep is immune to. Our findings represent essential benefits that agentic RAG brings when compared to fixed RAG paradigm. Nevertheless, we highlight the importance of wide-scale evaluation on the diverse set of datasets to make the final conclusions.

Author Contribution Statement

Sergey Sedov (Branch 1: Data Engineering & Indexing): Scraped and cleaned YouTube transcripts (filtered by creator, length, and date), generated the synthetic multi-hop Q&A dataset, and implemented indexing pipelines for BM25 and embedding-based retrieval. This branch also included implementation of LLM-scored chunk quality metrics.

Vedant Jagtap (Branch 2: Agentic Architecture & Tooling): Developed the agentic framework and retrieval tools for `grep`, BM25, and vector retrieval; implemented Judge-in-Process feedback hooks; and optimized the retrieval loop with context caching and batched generation for lower cost and better runtime.

Adhiraj Singh (Branch 3: Experimental Evaluation): Automated the 36-experiment condition grid, implemented and validated metric reporting (including success rate, retrieval precision, and trajectory statistics), and analyzed performance deltas between baseline conditions and process-feedback-augmented conditions.

The project was coordinated as three integrated workstreams, with each member owning a primary branch and collaborating during weekly integration meetings on cross-branch debugging, experiment design updates, and report synthesis.

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. SELF-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*.

Michael Krumdick, Charles Lovering, Varshini Reddy, Seth Ebner, and Chris Tanner. 2026. No free labels: Limitations of llm-as-a-judge without human grounding. *Preprint*, arXiv:2503.05061.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

Xuanlin Lin and 1 others. 2025. RAGCap-Bench: Benchmarking capabilities of LLMs in RAG. *arXiv preprint*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.

Manveer Singh Tamber, Forrest Sheng Bao, Chenyu Xu, and 1 others. 2025. Benchmarking LLM faithfulness in RAG with evolving leaderboards. *arXiv preprint*.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*.